

数字逻辑与处理器基础作业——汇编程序设计

2026 年 春

一、作业概述

本次作业要求同学们在 MARS 模拟器上，用 MIPS 汇编指令实现指定的算法解决问题，然后汇编，运行，调试，完成能够解决给定问题的汇编程序设计。本次作业的目的在于理解汇编语言如何完成高级语言描述的算法，了解 MIPS 处理器的硬件结构如何实现指令的需求，同时学会如何编写调试汇编程序。

本次作业分为两个部分：基础练习和实战应用。基础练习部分中，会对 MIPS 常见汇编操作进行练习，学习 MIPS 汇编程序编写的基本方式。实战应用部分中，需要根据给出 2 个不同算法对应的伪代码，编写算法功能完整的汇编程序。

二、提交要求

本次作业的提交要求如下：

作业在 2026 年 6 月 14 日(第十六周周日) 23:59 前用一个压缩包提交到网络学堂，压缩包名称：“学号_姓名.xxx”。“xxx”为压缩包格式，必须用 zip /rar / 7z 格式。压缩包解压后需要包含：

一个“实验报告.pdf”文件，内容要求言简意赅，按照作业内容分为两个部分：基础练习部分简述程序测试功能即可，实战应用部分只需要介绍寄存器和变量之间的分配关系、过程调用时进行的操作和最终程序输出即可；

一个“exp1”文件夹，其中包含：一个“exp1_1.asm”汇编代码文件，一个“exp1_2.asm”汇编代码文件，一个“exp1_3.asm”汇编代码文件和一个“exp1_4.asm”汇编代码文件，总计 4 个文件；

一个“exp2/asm”文件夹（asm文件夹在exp2下），其中包含：一个“exp2_1.asm”汇编代码文件和一个“exp2_2.asm”汇编代码文件，总计 2 个文件。

上述所有文件名（含扩展名）中出现的字母**均要求小写**。

注：本次作业中的部分成绩会由评测脚本直接给出。**如果因为提交格式错误导致评测脚本无法正常找到代码进行评测，相应部分的分数会被扣除。**

三、注意事项

1. 本次实验中所有的 MIPS 代码均提供了相应的C代码/C++/伪代码（在 exp1 和 exp2 文件夹中），需要和所提供的代码对应，**不可另外使用其他 C/C++/伪代码**。
2. 本次作业须独立完成，**严禁任何形式的抄袭**。本次作业将进行查重（查重的范围包括但不限于同届和往届学生的代码及网络开源代码等）。如查重过程中一旦发现问题，并被认定为抄袭，则课程等级记为**F**并上报，并按照学校相关规章制度严肃处理。
3. 作业中**若使用AI辅助完成题目，必须明确标注本题由 AI 辅助完成，并说明AI提供商及模型名称（例如：OpenAI-GPT-5 Mini）**。同时，**需在附录中以截图形式提供与AI的完整对话记录**。若未标注或未提供完整对话，一旦查重发现与他人内容高度相似且无法进行合理解释，将由课组教师和全体助教投票判定，若票数超过 2/3，视为抄袭处理。学生如有异议，可提出申诉，申诉结果由学校教务部门审议并作出最终判定。

四、负责助教联系方式

如果对本次作业有任何疑问或建议，可以联系本次作业负责助教：

陈泽恺: zk-chen21@mails.tsinghua.edu.cn

陈仲豪: czh23@mails.tsinghua.edu.cn

第零部分 承诺书

在提交的实验报告首页，应附上已填写完成的承诺书截图。承诺书内容如下：

本人郑重承诺：已按照大作业文档中“提交要求”部分的说明，对所有相关文件进行整理，并按规定格式进行压缩打包提交。若因文件格式、命名或提交方式等不符合要求而导致丢分，本人愿承担全部责任。

同时，作业中如涉及使用人工智能（AI）生成或辅助完成的内容，均已在对应题目处明确标注所使用的 AI 提供商及模型名称，并在实验报告的附录中以截图形式提供与 AI 的完整对话记录，确保内容真实、来源清晰、过程可追溯。

承诺人：_____

日 期：_____

第一部分 基础练习（4分）

练习 1-1：系统调用（exp1_1.asm）

练习使用 MARS 模拟器中的系统调用 `syscall`，使用 `syscall` 可以完成包括文件读写，命令行读写（标准输入输出），申请内存等辅助功能。系统调用基本的使用方法是

- 向 `$a*` 寄存中写入需要的参数（如果有）
- 向 `$v0` 寄存器中写入需要调用的 `syscall` 的编号
- 使用“`syscall`”指令进行调用
- 从 `$v0` 中读取调用的返回值（如果有）

更多具体的使用方法可以参照 MARS 模拟器的 Help 中的相关内容。
`exp1_1.cpp` 代码内容主要包括：

- 申请一个 8 byte 整数的内存空间。
- 从“`a.in`”读取两个整数。
- 向“`a.out`”写入这两个整数。
- 从键盘输入一个整数 `i`。
- `i = i + 10`。
- 向屏幕打印这个整数。

输入输出文件格式： 输入文件名为“`a.in`”，输出文件名为“`a.out`”，输入输出文件均使用二进制格式。文件中提供了“`a.in`”作为测试样例，包含 1 和 10 两个整数。

提示：对于打开文件的 MIPS 指令，只读对应 `flag = 0`，只写对应 `flag = 1`，可读可写对应 `flag = 2`。

练习 1-2: 循环, 分支 (exp1_2.asm)

用 MIPS 语言实现 exp1_2.cpp 中的功能并提交汇编代码, 尽量在代码中添加注释。exp1_2.cpp 代码内容主要包括:

- 将输入值 i 取相反数, 将输入值 j 取绝对值。(j 为非 0 整数)
- 从变量 i 开始, 循环 j 轮, 每轮 $i = i+1$

输出格式要求: 最终程序运行结束时**结果储存在寄存器 \$v0 中**。没有按要求输出将会酌情扣分。

练习 1-3: 数组, 指针 (exp1_3.asm)

用 MIPS 汇编指令实现 exp1_3.cpp 的功能并提交汇编代码, 尽量在代码中添加注释。exp1_3.cpp 代码内容主要包括:

- 输入数组 a 的长度 n
- 任意输入 n 个整数
- 将数组 a 中的元素逐个加 1, 并且逆序存储在 a 中
- 打印数组 a 的值

提示: MIPS 中系统调用 9 和 C 语言中的 `new` 作用类似。使用该指令开辟 n 个整数的空间, 传入参数为 $n*4$, 返回值为空间首地址。

练习 1-4: 函数调用 (exp1_4.asm)

本节提供了计算斐波那契数列的 C++ 语言代码 exp1_4.cpp, 请翻译成 MIPS 代码。

第二部分 实战应用（6分）

2.1 题目描述

给定一个长度为 N (N 为偶数) 的序列，问能否将其划分为两个长度为 $N/2$ 的严格递增子序列。

输入格式

一行，表示一组数据。

首先输入一个整数 N ，表示序列的长度。之后 N 个整数表示这个序列。

输出格式

对于数据，如果存在一种划分，则输出 1，否则输出 0。

输入输出样例 #1

输入 #1

6 3 1 4 5 8 7 ``

输出

1

【数据范围】

对于 100% 的数据，满足 $N \leq 2000$ 。

输入/输出方法

输入文件为命名为 test.dat 的二进制文件，具体输入格式可参考提供的测试文件 test.dat，其中包含“问题示例”部分所给出的例子对应的数据。

输出直接打印到屏幕，共 1 个整数

2.2 题目描述

树型网络是最节省材料的网络。所谓树型网络，是指一个无环的连通网络，网络中任意两个结点间有且仅有一条通信道路。网络中有一个结点是服务器，负责将信号直接或间接地发送到各终端机。如图上方，server 结点发出一个信号给结点 a 和 c，a 再转发给 b。如此，整个网络都收到这个信号了。

但是，实际操作中，信号从一个结点发到另一个结点，会出现信号强度的衰减。衰减量一般由线路长度决定。

如图下方，边上所标的数字为边的衰减量。假设从 server 出发一个强度为 4 个单位的信号，发到结点 a 后强度衰减为 $4-3=1$ 个单位。结点 a 再将其转发给结点 b。由于信号强度为 1，衰减量为 2，因此信号无法发送到 b。

一个解决这一问题的方法是，安装信号放大器。信号放大器的作用是将强度大于零的信号还原成初始强度（从服务器出发时的强度）。

上图中，若在结点 a 处安装一个信号放大器，则强度为 4 的信号发到 a 处，即被放大至 4。这样，信号就可以被发送到网络中的任意一个节点了。为了简化问题，我们假定每个结点只处理一次信号，当它第二次收到某个信号时，就忽略此信号。

你的任务是根据给出的树型网络，计算出最少需要安装的信号放大器数量。

输入格式

第一个一个整数 n，表示网络中结点的数量。（n 小于 20000）输出格式

第 2 到 n+1 行，每行描述一个节点的连接关系。其中第 i+1 行，描述的是结点 i 的连接关系：首先一个整数 k，表示与结点 i 相连的结点的数量。此后 2k 个数，每两个描述一个与结点 i 相连的结点，分别表示结点的编号（编号在 1 到 n 之间）和该结点与结点 i 之间的边的信号衰减量。结点 1

表示服务器。

最后一行，一个整数，表示从服务器上出发信号的强度。

输出格式

一个整数，表示要使信号能够传遍整个网络，需要安装的最少的信号放大器数量。

如果不论如何安装信号放大器，都无法使信号传遍整个网络，则输出0。

输入输出样例 #1

输入 #1

4

2 2 3 3 1

2 1 3 4 2

1 1 1

1 2 2

4

输出

1

【数据范围】

对于 100% 的数据，满足 $N \leq 2000$ 。

输入/输出方法

输入文件为命名为`test.dat`的二进制文件，具体输入格式可参考提供的测试文件 `test.dat`，其中包含“问题示例”部分所给出的例子对应的数据。

请勿在输出末尾额外输出空行。

请勿输出额外的输入输出提示。

实验内容

本部分作业提供了题解的2个算法的伪代码：`exp2_1.txt`与`exp2_2.txt`，相应代码位于 `exp2/example/`路径下。

需要各位同学在指定路径文件中完成相应的代码。提供的汇编文件名与伪代码文件名对应。完成后的代码应该位于 `exp2/asm/`路径下。

汇编代码部分中，需要有必要的注释来对代码进行解释说明，以便于助教评分。

本部分作业对相应算法原理的掌握不做要求，相应算法原理可以参考《数据与算法》等课程的相应课件。

注：请注意编写汇编代码时需要与伪代码方法对应，不需要逐行匹配，但大体程序结构应该相似（比如循环与递归两种写法之间不能串用）。如果助教批改过程中发现未按照伪代码给出的示范将扣除部分分数。同时按照伪代码的实现应该达到相近的计算复杂度，如果程序运行时间过长，评测中会自动扣除这部分分数。

以下内容附加题，请各位同学根据自身情况选择性完成，不完成不会减分，完成则会视具体情况加分（本次大作业满分10分，完成附加题加分后大作业二总分不超过上限13分）

第三部分 附加题目（5分）

近年来，RISC-V指令集因为其开放性、精简性、可扩展性以及优异的性能而广受欢迎，MIPS所属公司也在之前宣布要转而投入RISC-V。

下面给出了32bit下RISC-V指令集中基础指令集的指令分类与指令特征，请仔细思考、查阅资料，尝试完成以下题目：

分析这几种RISC-V指令类型在处理器中的数据通路、控制信号和MIPS的几种指令相比有何异同。然后仿照处理器、流水线几节课程PPT中的画法，画出单周期、多周期、流水线设计下的RISC-V处理器中，几种与MIPS处理器有明显不同的指令的数据通路、控制信号示意图。

（默认各个单元按课程中所教授的处理器结构为基础，必要的话可以增删修改单元；请在PowerPoint中完成绘图并提交文件，可参考附件）

RISC-V中的基础指令分类：

Name (Field Size)	Field						Comments
	7 bits	5 bits	5 bits	3 bits	5 bits	7 bits	
R-type	funct7	rs2	rs1	funct3	rd	opcode	Arithmetic instruction format
I-type	immediate[11:0]		rs1	funct3	rd	opcode	Loads & immediate arithmetic
S-type	immed[11:5]	rs2	rs1	funct3	immed[4:0]	opcode	Stores
SB-type	immed[12,10:5]	rs2	rs1	funct3	immed[4:1,11]	opcode	Conditional branch format
UJ-type	immediate[20,10:1,11,19:12]				rd	opcode	Unconditional jump format
U-type	immediate[31:12]				rd	opcode	Upper immediate format

Example instruction	Instruction name	Meaning
ld x1,80(x2)	Load doubleword	$\text{Regs}[x1] \leftarrow \text{Mem}[80 + \text{Regs}[x2]]$
lw x1,60(x2)	Load word	$\text{Regs}[x1] \leftarrow_{64} \text{Mem}[60 + \text{Regs}[x2]]_0^{32} \text{ ##}$ $\text{Mem}[60 + \text{Regs}[x2]]$
lwu x1,60(x2)	Load word unsigned	$\text{Regs}[x1] \leftarrow_{64} 0^{32} \text{ ##} \text{Mem}[60 + \text{Regs}[x2]]$
lb x1,40(x3)	Load byte	$\text{Regs}[x1] \leftarrow_{64} (\text{Mem}[40 + \text{Regs}[x3]]_0^{56} \text{ ##}$ $\text{Mem}[40 + \text{Regs}[x3]])$
lbu x1,40(x3)	Load byte unsigned	$\text{Regs}[x1] \leftarrow_{64} 0^{56} \text{ ##} \text{Mem}[40 + \text{Regs}[x3]]$
lh x1,40(x3)	Load half word	$\text{Regs}[x1] \leftarrow_{64} (\text{Mem}[40 + \text{Regs}[x3]]_0^{48} \text{ ##}$ $\text{Mem}[40 + \text{Regs}[x3]])$
flw f0,50(x3)	Load FP single	$\text{Regs}[f0] \leftarrow_{64} \text{Mem}[50 + \text{Regs}[x3]] \text{ ##} 0^{32}$
fld f0,50(x2)	Load FP double	$\text{Regs}[f0] \leftarrow_{64} \text{Mem}[50 + \text{Regs}[x2]]$
sd x2,400(x3)	Store double	$\text{Mem}[400 + \text{Regs}[x3]] \leftarrow_{64} \text{Regs}[x2]$
sw x3,500(x4)	Store word	$\text{Mem}[500 + \text{Regs}[x4]] \leftarrow_{32} \text{Regs}[x3]_{32..63}$
fsw f0,40(x3)	Store FP single	$\text{Mem}[40 + \text{Regs}[x3]] \leftarrow_{32} \text{Regs}[f0]_{0..31}$
fsd f0,40(x3)	Store FP double	$\text{Mem}[40 + \text{Regs}[x3]] \leftarrow_{64} \text{Regs}[f0]$
sh x3,502(x2)	Store half	$\text{Mem}[502 + \text{Regs}[x2]] \leftarrow_{16} \text{Regs}[x3]_{48..63}$
sb x2,41(x3)	Store byte	$\text{Mem}[41 + \text{Regs}[x3]] \leftarrow_8 \text{Regs}[x2]_{56..63}$

Figure A.25 The load and store instructions in RISC-V. Loads shorter than 64 bits are available in both sign-extended and zero-extended forms. All memory references use a single addressing mode. Of course, both loads and stores are available for all the data types shown. Because RV64G supports double precision floating point, all single precision floating point loads must be aligned in the FP register, which are 64-bits wide.

Example instruction	Instruction name	Meaning
add x1,x2,x3	Add	$\text{Regs}[x1] \leftarrow \text{Regs}[x2] + \text{Regs}[x3]$
addi x1,x2,3	Add immediate unsigned	$\text{Regs}[x1] \leftarrow \text{Regs}[x2] + 3$
lui x1,42	Load upper immediate	$\text{Regs}[x1] \leftarrow 0^{32} \text{ ##} 42 \text{ ##} 0^{12}$
sll x1,x2,5	Shift left logical	$\text{Regs}[x1] \leftarrow \text{Regs}[x2] \ll 5$
slt x1,x2,x3	Set less than	$\text{if } (\text{Regs}[x2] < \text{Regs}[x3])$ $\text{Regs}[x1] \leftarrow 1 \text{ else } \text{Regs}[x1] \leftarrow 0$

Figure A.26 The basic ALU instructions in RISC-V are available both with register-register operands and with one immediate operand. LUI uses the U-format that employs the rs1 field as part of the immediate, yielding a 20-bit immediate.

Example instruction	Instruction name	Meaning
jal x1,offset	Jump and link	$\text{Regs}[x1] \leftarrow \text{PC} + 4; \text{PC} \leftarrow \text{PC} + (\text{offset} \ll 1)$
jalr x1,x2,offset	Jump and link register	$\text{Regs}[x1] \leftarrow \text{PC} + 4; \text{PC} \leftarrow \text{Regs}[x2] + \text{offset}$
beq x3,x4,offset	Branch equal zero	$\text{if } (\text{Regs}[x3] == \text{Regs}[x4]) \text{PC} \leftarrow \text{PC} + (\text{offset} \ll 1)$
bgt x3,x4,name	Branch not equal zero	$\text{if } (\text{Regs}[x3] > \text{Regs}[x4]) \text{PC} \leftarrow \text{PC} + (\text{offset} \ll 1)$

Figure A.27 Typical control flow instructions in RISC-V. All control instructions, except jumps to an address in a register, are PC-relative.

注：RISC-V中没有延迟槽设计

参考资料:

Hennessy, J. L., & Patterson, D. A. (2011). Computer architecture: a quantitative approach. Elsevier.